Rockset Real-Time Database Architecture

The Aggregator-Leaf-Tailer (ALT) Topology & RocksDB-Cloud Decoupling

< 1 Second	**< 50 ms**
STREAMING DATA FRESHNESS (INGEST TO QUERY)	P95 ANSI SQL ANALYTICAL QUERY LATENCY

1. The Real-Time Operational Analytics Gap

Modern application experiences—such as real-time personalization, fraud detection, and operational dashboards—require running complex SQL queries with joins and aggregations on fast-mutating, semi-structured JSON streams. Traditional database systems fail to satisfy these dual constraints:

- **Data Warehouses (Snowflake, BigQuery):** Built for batch pipelines. Ingestion latency ranges from 5 minutes to hours, and query latency and costs cannot support live user-facing applications.
- **Relational OLTP (PostgreSQL, MySQL):** High lock contention under streaming writes; lack vectorized columnar engines, causing full-table analytical scans to exhaust CPU.
- **Search Engines (Elasticsearch):** Highly optimized for text token search, but lack ANSI SQL joins and exhibit severe memory bloat and slow numeric aggregations.

2. The Aggregator-Leaf-Tailer (ALT) Architecture

Rockset decouples ingest, storage indexing, and query serving into three specialized distributed tiers:

Tier	Statefulness	Primary Responsibilities	Scaling Vector
Tailer Tier	Stateless	Continuous pull from streaming sources (Kafka/CDC/S3); JSON shredding; Smart Schema inference.	Scales with **ingestion throughput** (MB/s).
Leaf Tier	Stateful (Sharded)	Hosts embedded RocksDB engines; maintains the Converged Index; manages local NVMe read caches.	Scales with **data volume & active working set**.
Aggregator Tier	Stateless	SQL parsing, cost-based optimization, distributed plan execution, scatter-gather, and join merging.	Scales with **query concurrency & complexity**.

3. The Tailer: Smart Schemas on Dynamic JSON

Unlike traditional databases requiring `ALTER TABLE` migrations, Tailers dynamically parse deeply nested JSON. Documents are shredded into key-value pairs without up-front schema definitions, dynamically binding types per value and storing sparse structures efficiently.

4. The Leaf Tier: Stateful Shards Powered by RocksDB

Each collection is divided into logical shards replicated across Leaf nodes. Every shard replica runs an embedded instance of **RocksDB**, leveraging Log-Structured Merge-trees (LSM-trees) to turn random document mutations into high-speed sequential writes to memory (`MemTable`) and Write-Ahead Logs (WAL).

5. Storage-Compute Decoupling via RocksDB-Cloud

In vanilla RocksDB, data is tied to local disks. Rockset engineered **RocksDB-Cloud**, treating cloud object storage (AWS S3) as the durable shared medium:

The RocksDB-Cloud Storage Hierarchy
[Active MemTable (RAM)] → [Local Ephemeral NVMe SSD Block Cache] → [Immutable SSTables in Cloud Object Store (AWS S3)]

When a Leaf flushes an immutable SSTable, it is asynchronously uploaded to S3. Local NVMe drives act strictly as an eviction-driven cache. If a Leaf node crashes, a replacement node spins up in seconds, downloads lightweight metadata manifests from S3, and serves live queries immediately via zero-copy clone mechanisms.

6. Eliminating Tail Latency with Remote Compaction

In traditional LSM engines, background compactions (merging SST levels $L_i \to L_{i+1}$) consume 30–50% of CPU and IOPS bandwidth, creating "write stalls" and query tail spikes.

```
Workload_query ∩ Workload_compaction = ∅
```

Rockset delegates compactions to an independent, autoscaling fleet of **Serverless Remote Compactors**. The compactors read SSTs from S3, perform deduplication, write new SST runs to S3, and notify the Leaf nodes via lightweight metadata updates, keeping Leaf query compute 100% free of compaction noise.

7. The Aggregator: Distributed SQL Query Planning

Aggregators receive incoming ANSI SQL queries and compile them into physical execution DAGs:

```
-- Example Real-time Operational Analytics Query
SELECT
    d.category,
    COUNT(*) AS total_orders,
    SUM(d.price * d.quantity) AS gross_revenue
FROM orders o
JOIN items d ON o.item_id = d.id
WHERE o.created_at > NOW() - INTERVAL 1 HOUR
GROUP BY d.category
ORDER BY gross_revenue DESC LIMIT 10;
```

The Aggregator pushes filtering and partial aggregations down to Leaf nodes. Leaves execute vectorized SIMD scans over their columnar cache blocks, returning compressed intermediate record batches. The Aggregator performs the final hash join, sorting, and client formatting.

8. Empirical System Performance Benchmarks

Metric Dimension	Traditional Data Warehouse	Search Engine (Elastic)	Rockset ALT Architecture
Ingestion to Query Latency	5 – 30 minutes (Micro-batch)	1 – 5 seconds	**< 1 second (Continuous)**
Analytical Query (p95)	1.2 – 4.5 seconds	350 – 800 ms (High GC)	**22 – 48 ms**
Complex SQL Joins	Fully supported	Unsupported (Requires pre-join)	**Full ANSI SQL Support**
Recovery from Node Loss	Minutes (Cluster re-balance)	Tens of minutes (Replica copy)	**< 5 seconds (S3 metadata clone)**

9. Architectural Lessons & Production Guardrails

1. Isolate Compute for Predictable Latency
Separating stateless query coordinators (Aggregators) from storage nodes (Leaves) guarantees that complex analytical queries cannot crash or stall the live ingestion pipeline.

2. Decouple Durability from Serving State
Treating local SSDs as caches and relying on cloud object stores for durability converts node failures into trivial metadata re-attachment events.